

CH03 — 프롬프트 엔지니어링 핵심

Part 01 · CH03 — 프롬프트 엔지니어링 핵심 (4h)

이 챕터의 목표 "같은 LLM이라도 어떻게 말을 거느냐에 따라 결과가 완전히 달라진다"는 것을 직접 체감합니다. Zero-shot / Few-shot / CoT 세 가지 전략을 비교하고, **OpenCV** 탐지 결과를 **LLM**이 이해할 수 있는 프롬프트로 변환하는 핵심 함수를 완성합니다. 이 챕터의 결과물이 Part 01 실습 스크립트 `step01_llm_from_opencv.py`의 핵심입니다.

학습 목표

#	학습 내용	난이도
1	좋은 프롬프트의 4요소 — 역할 / 맥락 / 지시 / 출력 형식	☆☆
2	Zero-shot / Few-shot / CoT 기법 비교 실습	☆☆
3	OpenCV 탐지 결과를 프롬프트 텍스트로 변환하기	☆☆☆
4	구조화된 JSON 출력 프롬프트 작성	☆☆☆
5	환각 방지 프롬프트 패턴	☆☆

프롬프트 엔지니어링이란?

LLM은 동일한 질문이라도 어떻게 말을 거느냐에 따라 완전히 다른 답변을 냅니다.

비유: 같은 요리사에게 "맛있는 것 해줘" vs "오늘 손님이 채식주의자이고 알레르기가 있으니 두부로 만든 고단백 메인 요리를 플레이팅까지 신경 써서 만들어줘"

프롬프트 엔지니어링은 **LLM**에게 말 거는 방식을 설계하는 기술입니다. 코드 수정 없이 프롬프트만 바꿔서 결과를 크게 개선할 수 있기 때문에, 실무에서 가장 빠른 성능 향상 수단입니다.

1. 좋은 프롬프트의 4요소

잘 동작하는 프롬프트에는 공통된 구조가 있습니다.

[역할]	당신은 어떤 AI인가?	
[맥락]	어떤 상황에서 쓰이는가?	
[지시]	무엇을 해야 하는가?	
[출력 형식]	어떤 형태로 답해야 하는가?	
[데이터]	분석할 실제 데이터	

4요소가 빠질수록 LLM은 빈칸을 자기 마음대로 채웁니다. 채워야 할 빈칸이 많을수록 환각(Hallucination)이 발생할 가능성이 높아집니다.

예제 01 — 4요소 프롬프트 조립기

```
# -----
# 예제 01: 좋은 프롬프트의 4요소 - 조립 함수
# - 프롬프트를 구성 요소별로 명시적으로 분리해서 관리합니다
# - API 키 없이 실행 가능합니다
# -----

def build_full_prompt(
    role: str,           # 역할: AI의 정체성
    context: str,        # 맥락: 어떤 상황에서 쓰이는지
    instruction: str,    # 지시: 무엇을 해야 하는지
    output_format: str,  # 출력 형식: 어떤 형태로 답해야 하는지
    data: str,          # 데이터: 분석할 실제 입력값
) -> str:
    """
```

프롬프트의 4요소를 명시적으로 조립합니다.

각 섹션을 분리해서 관리하면:

1. 수정이 쉽다 (역할만 바꾸거나, 형식만 바꾸거나)
2. 재사용이 쉽다 (데이터만 바꿔서 여러 상황에 활용)
3. 문제 원인 파악이 쉽다 (어느 섹션이 문제인지 알 수 있음)

####

.strip()은 앞뒤 공백/줄바꿈을 제거합니다

```
return "\n\n".join([
    f"[역할]\n{role.strip()}",
    f"[맥락]\n{context.strip()}",
    f"[지시]\n{instruction.strip()}",
    f"[출력 형식]\n{output_format.strip()}",
    f"[탐지 데이터]\n{data.strip()}",
])
```

— 각 요소 정의 —————

```
ROLE = (
    "당신은 AI 기반 CCTV 통합 보안 관제 시스템입니다. "
    "OpenCV 객체 탐지 결과를 분석하여 보안 위협을 판단합니다."
)
```

```
CONTEXT = (
    "분석 대상: 물류 창고 단지. "
    "야간·심야 시간대 무단 접근 사례가 연간 12건 발생. "
    "주요 위협 유형: 절도(67%), 기물파손(20%), 기타(13%). "
    "보안 등급: A급 (최고 수준)"
)
```

```
INSTRUCTION = (
    "아래 탐지 데이터를 분석하여 위험도를 판단하세요.\n"
    "– 탐지 데이터에 없는 정보는 절대 추가하지 마세요\n"
    "– 확인되지 않은 행동은 '확인 불가'로 표시하세요\n"
    "– 판단 근거는 반드시 탐지 데이터에 근거해야 합니다"
)
```

```
OUTPUT_FORMAT = """{
    "risk_level": "정상" | "주의" | "위험",
    "confidence": 0.0~1.0 (이 판단에 대한 확신도),
    "reason": "근거 (탐지 데이터 기반, 한 문장, 한국어)",
    "action": "즉시 조치 사항 (한 문장, 한국어)"
}"""
```

```
DATA = (
    "[탐지 시각] 2025-04-04 02:13 (심야)\n"
    "[탐지 위치] 창고 출입구\n"
    "[객체 요약] person 2명, car 1대\n"
    " [1] person | 신뢰도: 91% | 크기: 80×270px | 위치: 좌측\n"
    " [2] person | 신뢰도: 87% | 크기: 80×265px | 위치: 중앙\n"
    " [3] car | 신뢰도: 95% | 크기: 230×200px | 위치: 좌측"
)
```

— 조립 및 출력 —————

```
prompt = build_full_prompt(ROLE, CONTEXT, INSTRUCTION, OUTPUT_FORMAT, DATA)
```

```
print("=== 4요소 완성형 프롬프트 ===\n")
print(prompt)
print(f"\n{'-' * 50}")
print(f"📏 프롬프트 길이: {len(prompt)}자")
print(f"📊 예상 입력 토큰: 약 {int(len(prompt) * 0.6)}개")
print()
print("💡 각 섹션이 분리되어 있어:")
print("  – 위치만 바꾸려면 CONTEXT 한 줄만 수정")
print("  – 출력 필드를 추가하려면 OUTPUT_FORMAT만 수정")
print("  – 탐지 데이터만 교체하면 다른 프레임에 재사용 가능")
```

실행 결과:

=== 4요소 완성형 프롬프트 ===

[역할]

당신은 AI 기반 CCTV 통합 보안 관제 시스템입니다. OpenCV 객체 탐지 결과를 분석하여 보안 위협을 판단합니다.

[맥락]

분석 대상: 물류 창고 단지. 야간·심야 시간대 무단 접근 사례가 연간 12건 발생. 주요 위협 유형: 절도(67%), 기물파손(20%), 기타(13%). 보안 등급: A급 (최고 수준)

[지시]

아래 탐지 데이터를 분석하여 위험도를 판단하세요.

- 탐지 데이터에 없는 정보는 절대 추가하지 마세요
- 확인되지 않은 행동은 '확인 불가'로 표시하세요
- 판단 근거는 반드시 탐지 데이터에 근거해야 합니다

[출력 형식]

```
{
  "risk_level": "정상" | "주의" | "위험",
  "confidence": 0.0~1.0 (이 판단에 대한 확신도),
  "reason": "근거 (탐지 데이터 기반, 한 문장, 한국어)",
  "action": "즉시 조치 사항 (한 문장, 한국어)"
}
```

[탐지 데이터]

[탐지 시각] 2025-04-04 02:13 (심야)

...

2. Zero-shot / Few-shot / CoT 비교

세 가지 기법의 차이를 같은 탐지 상황으로 직접 비교합니다.

2-1. Zero-shot — "예시 없이 물어보기"

예시를 전혀 주지 않고 지시만으로 답변을 요청하는 방식입니다.

장점: 프롬프트가 짧다 → 토큰 적게 사용
단점: 형식이 제각각 → 코드로 파싱하기 어렵다

```
# -----
# 예제 02: Zero-shot - 형식 불일치 문제 체험
# - 예시 없이 물어보면 응답 형식이 들쭉날쭉합니다
# - API 키 없이 실행 가능합니다 (문제 상황 시뮬레이션)
# -----

# Zero-shot 프롬프트 - 지시만 있고 예시가 없음
zero_shot_prompt = """창고에서 사람 2명이 탐지됐습니다. 위험한가요?"""

# 같은 프롬프트에 대해 GPT가 실제로 내놓는 다양한 응답 형식들 (mock)
# → 매번 다른 형식으로 답변해서 코드 파싱이 불가능
mock_responses = [
    "네, 창고에 사람 2명이 있으면 위험할 수 있습니다.",
    "It depends on the context and time of day.",
    "위험도: 중간. 상황을 더 파악해야 합니다.",
    "⚠️ 주의 필요! 경비 확인이 권장됩니다.",
    '{"danger": true, "level": "medium"}',
]

# 자유 텍스트
# 영어로 답변
# 형식 불규칙
# 이모지 포함
# 다른 JSON 키

print("=== Zero-shot - 응답 형식 불일치 문제 ===\n")
print(f"프롬프트: \"{zero_shot_prompt}\\n\\n")
print("같은 프롬프트에서 나올 수 있는 응답들:")
print("-" * 55)

for i, resp in enumerate(mock_responses, 1):
    # 각 응답을 파싱 시도
    import json
    try:
        parsed = json.loads(resp)
```

```

    parseable = "✅ 파싱 가능"
    # risk_level 키가 있는지 확인
    has_risk = "✅" if "risk_level" in parsed else "❌ risk_level 키 없음"
except json.JSONDecodeError:
    parseable = "❌ 파싱 불가 (JSON 아님)"
    has_risk = "❌"

print(f"   응답 {i}: {resp[:50]}")
print(f"           → {parseable}")
print()
```

```

print("결론: 예시 없이 물으면 형식이 제각각")
print("           → 자동화 시스템에서 코드로 처리하기 불가능")
print()
print("Zero-shot이 유용한 경우:")
print("  - 빠른 테스트/프로토타입")
print("  - 응답 형식이 중요하지 않은 탐색적 질문")
print("  - Few-shot 예시 준비가 어려울 때")
```

실행 결과:

=== Zero-shot - 응답 형식 불일치 문제 ===

프롬프트: "창고에서 사람 2명이 탐지됐습니다. 위험한가요?"

같은 프롬프트에서 나올 수 있는 응답들:

-
- 응답 1: 네, 창고에 사람 2명이 있으면 위험할 수 있습니다.
→ ❌ 파싱 불가 (JSON 아님)
- 응답 2: It depends on the context and time of day.
→ ❌ 파싱 불가 (JSON 아님)
- 응답 3: 위험도: 중간. 상황을 더 파악해야 합니다.
→ ❌ 파싱 불가 (JSON 아님)
- 응답 4: ⚠️ 주의 필요! 경비 확인이 권장됩니다.
→ ❌ 파싱 불가 (JSON 아님)
- 응답 5: {"danger": true, "level": "medium"}
→ ✅ 파싱 가능
→ ❌ risk_level 키 없음

2-2. Few-shot — "예시를 보여주고 따라하게 하기"

원하는 입출력 쌍 예시를 2~5개 보여주는 방식입니다.

- 장점: 형식이 일관됨 → 코드로 파싱 가능
"정상/주의/위험" 기준을 직접 보여줄 수 있음
- 단점: 프롬프트 길이 증가 → 토큰 비용 상승

```
# -----
# 예제 03: Few-shot - 예시를 보여주고 따라하게 하기
# - 정상/주의/위험 각 1개씩 예시를 주면 기준이 명확해집니다
# - API 키 없이 실행 가능합니다
# -----

def build_few_shot_prompt(examples: list, current_input: str) -> str:
    """
    Few-shot 프롬프트를 조립합니다.

    Args:
        examples      : [{"input": ..., "output": ...}, ...] 형태의 예시 목록
        current_input : 이번에 분석할 탐지 상황

    Returns:
        str: 조립된 Few-shot 프롬프트
```

```

"""
# 프롬프트 시작 — 예시를 참고하라는 안내
lines = [
    "아래 예시와 동일한 JSON 형식으로 분석하세요.",
    "예시에 나온 위험도 기준을 따르세요.",
    "",
]

# 예시 삽입
for i, ex in enumerate(examples, 1):
    lines.append(f"[예시 {i}]")
    lines.append(f"입력: {ex['input']}")
    lines.append(f"출력: {ex['output']}")
    lines.append("") # 예시 사이 빈 줄

# 실제 분석 대상
lines.append("[분석 대상]")
lines.append(f"입력: {current_input}")
lines.append("출력:") # LLM이 이어서 채우도록 유도

return "\n".join(lines)

# — Few-shot 예시 3개 (정상/주의/위험 균형) —
# 예시는 반드시 실제로 일어날 수 있는 상황으로 구성합니다
# 각 위험도별 기준이 명확히 드러나도록 선택합니다
examples = [
    {
        # 정상 예시: 주간 + 정문 + 1명 → 출입 패턴
        "input": "2025-04-04 14:30 (주간) / 정문 앞 / person 1명",
        "output": (
            '{"risk_level": "정상", "confidence": 0.95, '
            '"reason": "주간 정문 1인 출입 - 정상 패턴", '
            '"action": "로그 기록"}'
        ),
    },
    {
        # 주의 예시: 야간 + 창고 외곽 + 1명 → 배회 가능
        "input": "2025-04-04 22:00 (야간) / 창고 외곽 / person 1명",
        "output": (
            '{"risk_level": "주의", "confidence": 0.80, '
            '"reason": "야간 창고 외곽 1인 배회 - 이상 패턴", '
            '"action": "경비 확인 요청"}'
        ),
    },
    {
        # 위험 예시: 심야 + 창고 출입구 + 3명 + 차량 → 침입 시도
        "input": "2025-04-04 02:13 (심야) / 창고 출입구 / person 3명, car 1대",
        "output": (
            '{"risk_level": "위험", "confidence": 0.92, '
            '"reason": "심야 핵심 구역 3인+차량 - 침입 시도 패턴", '
            '"action": "경찰 즉시 신고"}'
        ),
    },
]

# 분석할 새로운 상황
current = "2025-04-04 03:15 (심야) / 주차장 A구역 / person 2명, car 1대"

prompt = build_few_shot_prompt(examples, current)

print("=== Few-shot 프롬프트 ===\n")
print(prompt)
print()

# 토큰 수 계산 (한국어 0.6배 추정)
est_tokens = int(len(prompt) * 0.6)
print(f"{'-' * 50}")
print(f"📊 예상 입력 토큰: {est_tokens}개")
print()
print("💡 Few-shot 예시 선택 원칙:")
print("1. 정상/주의/위험 각 1개씩 - 기준이 고르게 학습됨")

```

```
print("    2. 실제 발생 가능한 상황으로 - 현실성 있는 판단 기준")
print("    3. 출력 형식은 반드시 동일하게 - 형식 일관성 보장")
```

실행 결과:

=== Few-shot 프롬프트 ===

아래 예시와 동일한 JSON 형식으로 분석하세요.
예시에 나온 위험도 기준을 따르세요.

[예시 1]

입력: 2025-04-04 14:30 (주간) / 정문 앞 / person 1명

출력: {"risk_level": "정상", "confidence": 0.95, "reason": "주간 정문 1인 출입 - 정상 패턴", "action": "로그 기록"}

[예시 2]

입력: 2025-04-04 22:00 (야간) / 창고 외곽 / person 1명

출력: {"risk_level": "주의", "confidence": 0.80, "reason": "야간 창고 외곽 1인 배회 - 이상 패턴", "action": "경비 확인 요청"}

[예시 3]

입력: 2025-04-04 02:13 (심야) / 창고 출입구 / person 3명, car 1대

출력: {"risk_level": "위험", "confidence": 0.92, "reason": "심야 핵심 구역 3인+차량 - 침입 시도 패턴", "action": "경찰 즉시 신고"}

[분석 대상]

입력: 2025-04-04 03:15 (심야) / 주차장 A구역 / person 2명, car 1대

출력:

2-3. CoT — "단계별로 생각하게 하기"

Chain-of-Thought: 최종 답을 바로 내리지 않고, 추론 과정을 단계별로 밟도록 유도합니다.

장점: 복잡한 판단에서 정확도가 크게 향상됨
왜 이런 판단을 내렸는지 근거가 남아 감사(Audit) 가능
단점: 출력 토큰이 많아짐 → 비용 증가, 응답 속도 느려짐

```
# -----
# 예제 04: CoT - 단계별 추론 프롬프트
# - "단계별로 생각해" 한 마디가 정확도를 크게 높입니다
# - API 키 없이 실행 가능합니다
# -----
```

```
def build_cot_prompt(detection_text: str) -> str:
    """
    CoT(Chain-of-Thought) 프롬프트를 생성합니다.
    각 분석 단계를 명시해서 LLM이 논리적으로 추론하도록 유도합니다.
    """
    return f"""다음 CCTV 탐지 결과를 단계별로 분석해주세요.
```

탐지 결과:
{detection_text}

단계별 분석 (각 단계를 순서대로 수행):

1단계 - 시간대 분류

탐지 시각이 아래 중 어디에 해당하는가?

주간(06~18시) / 야간(18~24시) / 심야(00~06시)

2단계 - 위치 위험도 평가

탐지 위치의 보안 민감도는?

공개 구역(정문, 주차장) / 제한 구역(창고 외곽) / 핵심 구역(창고 출입구, 서버실)

3단계 - 객체 조합 분석

탐지된 인원 수와 차량 유무가 의미하는 바는?

(예: 다수 인원 + 차량 = 운반 수단 대기 가능성)

4단계 - 위험 패턴 매칭

위 3가지를 종합했을 때 알려진 이상 패턴과 일치하는가?


```
(예: 심야 + 핵심 구역 + 다인 + 차량 = 고위험 패턴)

5단계 — 최종 판단
위험도(정상/주의/위험)와 권고 조치를 결정하고
반드시 아래 JSON으로 출력하세요:

{{"risk_level": "정상|주의|위험", "confidence": 0.0~1.0, "reason": "단계별 분석 기반 근거", "action": "즉시 조치"}}"""

detection = (
    "[탐지 시각] 2025-04-04 02:13 (심야)\n"
    "[탐지 위치] 창고 출입구\n"
    "[객체 요약] person 2명, car 1대"
)

cot_prompt = build_cot_prompt(detection)
print("=== CoT 프롬프트 ===\n")
print(cot_prompt)
print()

# GPT CoT 응답 예시 (mock)
mock_cot_response = """
1단계: 02:13 → 심야 (00~06시)

2단계: 창고 출입구 → 핵심 보안 구역 (물품 반출 경로)

3단계: person 2명 + car 1대
→ 2인 이상: 협력 범행 가능성
→ 차량: 물품 운반 수단 대기 시사

4단계: 심야 + 핵심 구역 + 2인 + 차량
→ 절도 침입 고위험 패턴과 일치

5단계:
{"risk_level": "위험", "confidence": 0.94, "reason": "심야 창고 출입구 2인+차량 탐지 - 절도 시도 패턴", "action": "경찰 즉시 신고 + 비상 알람"}
"""

print("GPT CoT 응답 예시:")
print("-" * 50)
print(mock_cot_response)
print()
print("💡 CoT의 장점: 각 단계 근거가 남아 '왜 위험으로 판단했는지' 추적 가능")
print("💡 CoT가 특히 유용한 상황:")
print("  - 경계선상의 애매한 상황 판단")
print("  - 감사(Audit) 로그가 필요한 보안 시스템")
print("  - 탐지 결과 항목이 많아 종합 판단이 필요한 경우")
```

2-4. 세 가지 기법 비교 — 언제 무엇을 쓸까

```
# -----
# 예제 05: Zero-shot / Few-shot / CoT 성능 비교 정리
# - 실제 API 응답 기반 mock으로 세 기법의 차이를 정리합니다
# -----

comparison = [
    {
        "method": "Zero-shot",
        "desc": "예시 없이 지시만",
        "tokens_in": 80,
        "tokens_out": 45,
        "parseable": False,
        "accurate": "보통",
        "response": "위험합니다. 새벽에 창고에 사람과 차가 있으면 수상합니다.",
        "best_for": "빠른 프로토타입, 탐색적 질문",
    },
    {
        "method": "Few-shot",
        "desc": "예시 3개 제공",
```

```

        "tokens_in": 280,
        "tokens_out": 60,
        "parseable": True,
        "accurate": "높음",
        "response": '{"risk_level": "위험", "reason": "심야 창고 2인+차량", "action": "경비 출동"}',
        "best_for": "이 강의 기본 사용 방법 ← 추천",
    },
    {
        "method": "CoT",
        "desc": "단계별 추론 유도",
        "tokens_in": 320,
        "tokens_out": 180,
        "parseable": True,
        "accurate": "매우 높음",
        "response": "1단계: 심야... 2단계: 핵심구역... → {"risk_level": "위험", ...}",
        "best_for": "경계선 판단, 감사 로그 필요 시",
    },
]

print("=" * 70)
print(" Zero-shot / Few-shot / CoT 비교")
print("=" * 70)
print()
print(f"   {'기법':<12} {'입력 토큰':>9} {'출력 토큰':>9} {'파싱 가능':>9} {'정확도':>9}")
print("   " + "-" * 55)

for c in comparison:
    parseable_str = "✅" if c["parseable"] else "❌"
    # 추천 항목에 화살표 표시
    marker = " ←★" if "추천" in c["best_for"] else "   "
    print(
        f"   {c['method']:<12} {c['tokens_in']:>9,} {c['tokens_out']:>9,} "
        f"{parseable_str:>9} {c['accurate']:>9}{marker}"
    )

print()
print("   [응답 내용 비교]")
print("   " + "-" * 55)
for c in comparison:
    print(f"\n   📌 {c['method']} ({c['desc']})")
    resp = c["response"]
    # 80자 이상이면 줄여서 표시
    if len(resp) > 80:
        print(f"       {resp[:80]}...")
    else:
        print(f"       {resp}")
    print(f"       → 추천 용도: {c['best_for']}")

print()
print("   [비용 대비 효과 정리]")
print("   -" * 28)
print(" Zero-shot : 토큰 가장 적음, 하지만 파싱 불가 → 자동화 불가")
print(" Few-shot  : 토큰 3.5배, 파싱 가능, 형식 일관 → ✅ 실무 기본")
print(" CoT       : 토큰 6배,   가장 정확, 근거 추적 → 고위험 판단용")
```

실행 결과:

=====				
Zero-shot / Few-shot / CoT 비교				
=====				
기법	입력 토큰	출력 토큰	파싱 가능	정확도
Zero-shot	80	45	❌	보통
Few-shot	280	60	✅	높음 ←★
CoT	320	180	✅	매우 높음
[응답 내용 비교]				
📌 Zero-shot (예시 없이 지시만)				

위험합니다. 새벽에 창고에 사람과 차가 있으면 수상합니다.
→ 추천 용도: 빠른 프로토타입, 탐색적 질문

🔥 Few-shot (예시 3개 제공)
{`"risk_level": "위험"`, `"reason": "심야 창고 2인+차량"`, `"action": "경비 출동"`}
→ 추천 용도: 이 강의 기본 사용 방법 ← 추천

🔥 CoT (단계별 추론 유도)
1단계: 심야... 2단계: 핵심구역... → {`"risk_level": "위험"`, ...}
→ 추천 용도: 경계선 판단, 감사 로그 필요 시

3. OpenCV 탐지 결과를 프롬프트 텍스트로 변환하기

이 챕터의 핵심 실습입니다. OpenCV가 만들어주는 탐지 결과 딕셔너리를 LLM이 이해하기 좋은 텍스트로 바꿉니다.

왜 변환이 필요한가?

```
# OpenCV가 만들어주는 탐지 결과 (수강생이 이미 만들 수 있는 형태)
detections = [
    {"class": "person", "bbox": [120, 80, 200, 350], "confidence": 0.91},
    {"class": "person", "bbox": [310, 95, 390, 360], "confidence": 0.87},
    {"class": "car", "bbox": [ 50, 200, 280, 400], "confidence": 0.95},
]
```

이걸 그냥 LLM에 넣으면?

```
# ❌ 나쁜 방법 - 딕셔너리를 그냥 문자열로 변환해서 전달
bad_prompt = f"이 탐지 결과를 분석해줘: {str(detections)}"
# → LLM이 bbox 숫자가 뭔지, confidence가 뭔지 맥락을 파악해야 함
# → 불필요한 파싱 부담이 LLM에게 생김 → 오류 가능성 증가
```

해결책: 사람이 읽기 쉬운 형태로 먼저 변환합니다.

[탐지 시각] 2025-04-04 02:13 (심야)
[탐지 위치] 주차장 A구역
[객체 요약] person 2명, car 1대

[1] person | 신뢰도: 91% | 크기: 80×270px | 위치: 좌측
[2] person | 신뢰도: 87% | 크기: 80×265px | 위치: 중앙
[3] car | 신뢰도: 95% | 크기: 230×200px | 위치: 좌측

예제 06 — format_detections() 핵심 함수 완성

```
# -----
# 예제 06: format_detections() - OpenCV 탐지 결과 → LLM 입력 텍스트
# - 커리큘럼 전체에서 핵심적으로 재사용되는 함수입니다
# - Part 07 YOLO 프로젝트에서도 이 함수를 그대로 사용합니다
# - API 키 없이 실행 가능합니다
# -----
```

```
def get_time_zone(hour: int) -> str:
    """
    시각(0~23)을 기준으로 시간대를 반환합니다.

    Args:
        hour (int): 0~23 사이의 시간 값

    Returns:
        str: "주간" | "야간" | "심야"
    """
    if 6 <= hour < 18:      # 오전 6시 ~ 오후 6시
        return "주간"
    elif 18 <= hour < 24:   # 오후 6시 ~ 자정
        return "야간"
    else:                   # 자정 ~ 오전 6시
        return "심야"
```

```

def format_detections(
    detections: list,      # OpenCV 탐지 결과 리스트
    timestamp: str,        # 탐지 시각 "YYYY-MM-DD HH:MM"
    location: str,         # 탐지 위치 문자열
    img_width: int = 640,  # 이미지 너비 (위치 표현에 사용, 기본 640px)
) -> str:
    """
    OpenCV 탐지 결과 딕셔너리 리스트를
    LLM이 이해하기 좋은 텍스트로 변환합니다.

    Args:
        detections : [{"class": str, "bbox": [x1,y1,x2,y2], "confidence": float}, ...]
        timestamp  : 탐지 시각 문자열
        location    : 탐지 위치 문자열
        img_width   : 이미지 너비 (화면 좌/중/우 위치 표현에 사용)

    Returns:
        str: LLM에 전달할 탐지 결과 텍스트

    Note:
        이 함수는 Part 07 YOLO 프로젝트에서도 그대로 사용됩니다.
        YOLO 결과도 동일한 {"class", "bbox", "confidence"} 구조를 가집니다.
    """

# — 1. 시간대 추출 —————
try:
    # "2025-04-04 02:13" → "02:13" → "02" → 2
    hour = int(timestamp.split(" ")[1].split(":")[0])
    # .split(구분자): 문자열을 구분자 기준으로 나눕니다
    # [1]: 두 번째 원소 (시간 부분) / [0]: 첫 번째 원소 (시 부분)
    time_zone = get_time_zone(hour)
except (IndexError, ValueError):
    # 타임스탬프 형식이 예상과 다를 때 - 에러 대신 기본값
    time_zone = "시간대 불명"

# — 2. 클래스별 개수 집계 —————
class_count: dict = {}
for d in detections:
    cls = d["class"]
    # .get(키, 기본값): 키가 없으면 기본값 반환
    class_count[cls] = class_count.get(cls, 0) + 1

# "person 2명, car 1대" 형태의 요약 문자열 생성
summary_parts = []
for cls, cnt in class_count.items():
    # 사람은 "명", 차량은 "대" 단위 사용
    unit = "명" if cls == "person" else "대"
    summary_parts.append(f"{cls} {cnt}{unit}")
summary = ", ".join(summary_parts)
# ", ".join(리스트): 리스트 원소를 ", "로 이어붙입니다

# — 3. 헤더 줄 조립 —————
lines = [
    f"[탐지 시각] {timestamp} ({time_zone})",
    f"[탐지 위치] {location}",
    f"[탐지 건수] 총 {len(detections)}건",
    f"[객체 요약] {summary}",
    "", # 빈 줄로 구분
]

# — 4. 개별 탐지 상세 —————
# 화면을 3구역으로 나눠서 위치를 텍스트로 표현
# (숫자 좌표만 있으면 LLM이 공간 위치를 파악하기 어렵기 때문)
zone_width = img_width // 3 # 구역 너비 = 전체 너비 / 3
# 640px 기준: 좌측(0~213), 중앙(214~426), 우측(427~640)

for i, d in enumerate(detections, 1):
    # bbox: [x1, y1, x2, y2]
    # x1, y1 = 좌상단 좌표 / x2, y2 = 우하단 좌표
    x1, y1, x2, y2 = d["bbox"]

```

```

width = x2 - x1    # 바운딩박스 너비
height = y2 - y1   # 바운딩박스 높이

# 중심 x 좌표로 좌/중앙/우 판단
center_x = (x1 + x2) // 2    # // 는 정수 나눗셈
if center_x < zone_width:
    h_position = "좌측"
elif center_x < zone_width * 2:
    h_position = "중앙"
else:
    h_position = "우측"

# 신뢰도 % 표현
confidence_pct = d["confidence"]
# :.0% : 소수를 퍼센트로, 소수점 없이 출력 (0.91 → "91%")

lines.append(
    f"  [{i}] {d['class']:8s} | "
    f"신뢰도: {confidence_pct:.0%} | "
    f"크기: {width}x{height}px | "
    f"위치: {h_position} "
    f"({좌상단 {x1},{y1} → 우하단 {x2},{y2}})"
)

return "\n".join(lines)

# — 테스트 실행 —————
# 커리큘럼에 나온 예제 데이터 그대로 사용
detections = [
    {"class": "person", "bbox": [120, 80, 200, 350], "confidence": 0.91},
    {"class": "person", "bbox": [310, 95, 390, 360], "confidence": 0.87},
    {"class": "car", "bbox": [50, 200, 280, 400], "confidence": 0.95},
]

result_text = format_detections(
    detections = detections,
    timestamp = "2025-04-04 02:13",
    location = "주차장 A구역",
)

print("=== format_detections() 출력 ===\n")
print(result_text)
print()
print("-" * 55)
print()

# 다양한 시간대 테스트
time_tests = [
    ("2025-04-04 08:00", "주간 출근 시간"),
    ("2025-04-04 20:30", "야간 퇴근 이후"),
    ("2025-04-04 03:15", "심야 취약 시간"),
]

simple_detection = [{"class": "person", "bbox": [200, 100, 280, 380], "confidence": 0.88}]

print("시간대 분류 테스트:")
for ts, desc in time_tests:
    text = format_detections(simple_detection, ts, "정문 앞")
    # 첫 줄만 추출해서 확인
    first_line = text.split("\n")[0]
    print(f"  {desc:15s} → {first_line}")

print()
print("💡 YOLO 연결 포인트:")
print("  YOLO 탐지 결과도 동일한 구조입니다:")
print('  yolo_result = {"class": "person", "bbox": [...], "confidence": 0.88}')
print("  → format_detections()에 그대로 넣을 수 있습니다 (Part 07 활용)")

```

실행 결과:

=== format_detections() 출력 ===

[탐지 시각] 2025-04-04 02:13 (심야)
[탐지 위치] 주차장 A구역
[탐지 건수] 총 3건
[객체 요약] person 2명, car 1대

[1] person	신뢰도: 91%	크기: 80×270px	위치: 좌측 (좌상단 120,80 → 우하단 200,350)
[2] person	신뢰도: 87%	크기: 80×265px	위치: 중앙 (좌상단 310,95 → 우하단 390,360)
[3] car	신뢰도: 95%	크기: 230×200px	위치: 좌측 (좌상단 50,200 → 우하단 280,400)

시간대 분류 테스트:

주간 출근 시간 → [탐지 시각] 2025-04-04 08:00 (주간)
야간 퇴근 이후 → [탐지 시각] 2025-04-04 20:30 (야간)
심야 취약 시간 → [탐지 시각] 2025-04-04 03:15 (심야)

4. 환각 방지 프롬프트 패턴

GPT는 탐지 데이터에 없는 정보도 그럴듯하게 만들어냅니다. 보안 시스템에서는 이것이 심각한 문제가 됩니다.

탐지 데이터: "사람 2명 + 차량 1대 탐지됨"

GPT 환각 응답 예시:

"이 사람들은 장갑을 끼고 창고 자물쇠를 따고 있습니다."
→ 탐지 데이터에는 없는 행동을 사실처럼 단정!

예제 07 — 환각 방지 프롬프트 비교

```
# -----
# 예제 07: 환각 방지 - 좋은 프롬프트 vs 나쁜 프롬프트
# - 제약 조건을 명시하는 것만으로 환각을 크게 줄일 수 있습니다
# - API 키 없이 실행 가능합니다
# -----

detection_text = (
    "[탐지 시각] 02:13 (심야)\n"
    "[탐지 위치] 창고 출입구\n"
    "[객체 요약] person 2명, car 1대"
)

# — ❌ 나쁜 프롬프트 (환각 유발) -----
bad_prompt = f"""다음 CCTV 상황을 분석해주세요.

{detection_text}

이 사람들이 무엇을 하고 있는지 설명하고 위험도를 판단해주세요."""

# 나쁜 프롬프트의 전형적인 GPT 환각 응답 (mock)
bad_response = (
    "이 사람들은 창고에서 물건을 훔치고 있습니다. "
    "차량은 도주용으로 준비되어 있으며, 장갑과 마스크를 착용하고 있습니다. "
    "위험도: 위험"
)
# 문제점:
# "물건을 훔치고 있다" → 탐지 데이터에 없는 행동 단정
# "장갑과 마스크" → 완전한 환각 (카메라 해상도로는 알 수 없음)

# — ✅ 좋은 프롬프트 (환각 방지) -----
good_prompt = f"""다음 CCTV 탐지 데이터만을 근거로 분석하세요.

{detection_text}

[제약 조건 - 반드시 준수]
- 탐지 데이터에 없는 행동, 의도, 소지품은 절대 언급하지 마세요
```

- 확인할 수 없는 사항은 반드시 "확인 불가"로 표시하세요
- "~하고 있습니다"처럼 단정하지 말고, "~가능성이 있음"으로 표현하세요
- 탐지된 것: 인원 수, 위치, 차량 유무, 시각뿐입니다

JSON으로만 답하세요:

```
{{
  "risk_level": "정상|주의|위험",
  "reason":      "탐지 데이터 기반 근거만 (추측 금지)",
  "unknown":     ["확인 불가 항목 목록"]
}}"""
```

좋은 프롬프트의 GPT 응답 예시 (mock)

```
good_response = """{
  "risk_level": "위험",
  "reason": "심야 시간대 핵심 보안 구역(창고 출입구)에서 2인+차량 탐지 - 이상 패턴",
  "unknown": ["인물의 행동 목적", "차량 내부 상황", "인물 신원 및 복장"]
}"""
```

— 비교 출력 —

```
print("=== 환각 방지 프롬프트 비교 ===\n")
print("[공통 탐지 데이터]")
print(detection_text)
print()
```

```
print("-" * 55)
print("\n❌ 나쁜 프롬프트:")
print(f"    {bad_prompt[-80:]}...")
print()
print("    GPT 환각 응답:")
print(f"    {bad_response}")
print()
print("    문제점:")
print("    • '물건을 훔치고 있다' → 탐지 데이터에 없는 행동을 사실로 단정")
print("    • '장갑과 마스크' → 카메라로 확인 불가능한 정보를 생성")
print("    • 이런 정보로 경찰 신고하면 허위 신고가 됩니다!")
```

```
print("\n-" * 28)
```

```
print("\n✅ 좋은 프롬프트 (제약 조건 명시):")
print("    [제약 조건 포함]")
print("    - '탐지 데이터에 없는 정보 언급 금지'")
print("    - '확인 불가 항목은 unknown 필드에 명시'")
print()
print("    GPT 근거 기반 응답:")
print(f"    {good_response}")
print()
print("    효과:")
print("    • 행동 추측 없이 '탐지 사실'만 근거로 판단")
print("    • 확인 불가 항목이 명시되어 경비원이 직접 확인")
print()
print("-" * 55)
print("\n💡 환각 방지 핵심 3가지:")
print("    1. '탐지 데이터에 없는 정보 금지'를 명시")
print("    2. '확인 불가'로 표시할 unknown 필드 설계")
print("    3. '단정' 대신 '가능성' 언어 사용 지시")
```

5. CH03 최종 실습 — step01_llm_from_opencv.py 완성

앞서 배운 모든 기법을 하나로 통합합니다. Few-shot + 환각 방지 + `format_detections()` + JSON 출력을 조합한 최종 버전입니다.

```
# —————
# step01_llm_from_opencv.py
# Part 01 최종 실습 파일 - OpenCV 탐지 결과 → LLM 위험도 분석
#
# 이 파일에서 정의한 함수들은 Part 02~07에서 그대로 재사용됩니다:
#   format_detections() → Part 02 LangChain 배치 처리
#   SYSTEM_PROMPT       → Part 03 RAG 파이프라인
#   analyze_frame()      → Part 05 FastAPI 엔드포인트 내부 로직
#   결과 JSON 구조       → Part 06 React 대시보드 데이터 형식
```

```

# -----

import os
import json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# -----
# 섹션 1: 핵심 변환 함수
# -----

def get_time_zone(hour: int) -> str:
    """시각을 주간/야간/심야로 분류합니다."""
    if 6 <= hour < 18: return "주간"
    elif 18 <= hour < 24: return "야간"
    else: return "심야"

def format_detections(
    detections: list,
    timestamp: str,
    location: str,
    img_width: int = 640,
) -> str:
    """
    OpenCV 탐지 결과 리스트 → LLM 입력용 텍스트로 변환.

    YOLO 프로젝트 연결 포인트:
        YOLO 결과를 {"class", "bbox", "confidence"} 형태로 정리하면
        이 함수에 그대로 넣을 수 있습니다.
    """
    try:
        hour = int(timestamp.split(" ")[1].split(":")[0])
        tz = get_time_zone(hour)
    except (IndexError, ValueError):
        tz = "시간대 불명"

    # 클래스별 개수 집계
    class_count: dict = {}
    for d in detections:
        c = d["class"]
        class_count[c] = class_count.get(c, 0) + 1

    # 요약 문자열
    summary = ", ".join(
        f"{cls} {cnt}{'명' if cls == 'person' else '대'}"
        for cls, cnt in class_count.items()
    )

    lines = [
        f"[탐지 시각] {timestamp} ({tz})",
        f"[탐지 위치] {location}",
        f"[탐지 건수] 총 {len(detections)}건",
        f"[객체 요약] {summary}",
        "",
    ]

    zone = img_width // 3
    for i, d in enumerate(detections, 1):
        x1, y1, x2, y2 = d["bbox"]
        cx = (x1 + x2) // 2
        pos = "좌측" if cx < zone else ("중앙" if cx < zone * 2 else "우측")
        lines.append(
            f"  [{i}] {d['class']:8s} | 신뢰도: {d['confidence']:.0%} | "
            f"크기: {x2-x1}x{y2-y1}px | 위치: {pos} "
            f"([좌상단 {x1},{y1} → 우하단 {x2},{y2}])"
        )

```



```
return "\n".join(lines)

# =====
# 섹션 2: Few-shot 예시 정의 (한 번만 정의, 반복 재사용)
# =====

# 정상/주의/위험 각 1개씩 - 균형 잡힌 Few-shot 예시
FEW_SHOT_EXAMPLES = [
    {
        "input": "2025-04-04 14:30 (주간) / 정문 앞 / person 1명",
        "output": '{"risk_level":"정상","confidence":0.95,"reason":"주간 정문 1인 출입 - 정상 패턴","action":"로그 기록","unknown":[]}',
    },
    {
        "input": "2025-04-04 22:00 (야간) / 창고 외곽 / person 1명",
        "output": '{"risk_level":"주의","confidence":0.80,"reason":"야간 창고 외곽 1인 배회 - 이상 패턴","action":"경비 확인 요청","unknown":["배회 목적"]}',
    },
    {
        "input": "2025-04-04 02:13 (심야) / 창고 출입구 / person 3명, car 1대",
        "output": '{"risk_level":"위험","confidence":0.92,"reason":"심야 핵심 구역 3인+차량 - 침입 시도 패턴","action":"경찰 즉시 신고","unknown":["인물 신원"]}',
    },
]

# Few-shot 예시를 system prompt에 삽입
_few_shot_block = "\n\n".join(
    f"입력 예시 {i}:\n{ex['input']}\n출력 예시 {i}:\n{ex['output']}"
    for i, ex in enumerate(FEW_SHOT_EXAMPLES, 1)
)

SYSTEM_PROMPT = f"""당신은 AI CCTV 보안 분석 시스템입니다.
OpenCV로 탐지된 객체 정보를 분석하여 반드시 JSON 형식으로만 답하세요.

[Few-shot 예시 - 이 형식과 기준을 따르세요]
{_few_shot_block}

[출력 JSON 스키마]
{{
    "risk_level": "정상" | "주의" | "위험",
    "confidence": 0.0~1.0,
    "reason": "탐지 데이터 기반 근거 (한 문장)",
    "action": "즉시 조치 사항 (한 문장)",
    "unknown": ["확인 불가 항목"]
}}

[제약 조건 - 환각 방지]
- 탐지 데이터에 없는 정보는 절대 추가하지 마세요
- 확인할 수 없는 항목은 unknown 배열에 넣으세요
- 다른 텍스트 없이 JSON만 출력하세요"""

# =====
# 섹션 3: 분석 함수
# =====

def analyze_frame(
    detections: list,
    timestamp: str,
    location: str,
) -> dict:
    """
    CCTV 프레임의 탐지 결과를 LLM으로 분석합니다.

    Args:
        detections : OpenCV 탐지 결과 리스트
        timestamp   : 탐지 시각
        location     : 탐지 위치

    Returns:
        dict: 위험도 분석 결과 JSON
    """
```

Note:

Part 05 FastAPI에서 이 함수를 그대로 엔드포인트 내부에 넣습니다.

POST /analyze/detections → analyze_frame() 호출

"""

1. OpenCV 결과 → LLM 입력 텍스트 변환

detection_text = format_detections(detections, timestamp, location)

2. user 메시지 조립

user_message = f"다음 탐지 결과를 분석하세요:\n\n{detection_text}"

3. API 호출 (Few-shot + JSON 출력 + 환각 방지 모두 적용)

```
response = client.chat.completions.create(
    model          = "gpt-4o",
    messages       = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user",   "content": user_message},
    ],
    max_tokens     = 300,
    temperature    = 0.0,          # 분석은 결정적으로
    response_format = {"type": "json_object"},
)
```

4. JSON 파싱

raw = response.choices[0].message.content

result = json.loads(raw)

5. 메타 정보 추가 (비용 추적, 디버깅용)

```
result["_meta"] = {
    "timestamp": timestamp,
    "location": location,
    "total_tokens": response.usage.total_tokens,
}
```

return result

```
# =====
# 섹션 4: 실행 및 결과 출력
# =====
```

```
def print_result(result: dict) -> None:
    """분석 결과를 가독성 좋게 출력합니다."""
    icons = {"정상": "🟢", "주의": "🟡", "위험": "🔴"}
    level = result.get("risk_level", "정상")
    icon = icons.get(level, "⚪")

    meta = result.get("_meta", {})

    print(f" {icon} 위험도 : {level} (확신도: {result.get('confidence', 0):.0%})")
    print(f" 📄 근거 : {result.get('reason', 'N/A')}")
    print(f" 🛑 조치 : {result.get('action', 'N/A')}")

    unknowns = result.get("unknown", [])
    if unknowns:
        print(f" ? 확인불가: {' , '.join(unknowns)}")

    print(f" 📊 토큰 : {meta.get('total_tokens', 'N/A')}개")
```

if __name__ == "__main__":

— 테스트 시나리오 3가지 —

정상/주의/위험 각각의 상황으로 함수 동작 검증

```
test_scenarios = [
    {
        "label": "시나리오 A - 정상 (낮 출입)",
        "timestamp": "2025-04-04 14:30",
        "location": "정문 앞",
        "detections": [
            {"class": "person", "bbox": [200, 100, 280, 380], "confidence": 0.93}
        ]
    }
]
```



```

    ],
  },
  {
    "label":      "시나리오 B - 주의 (야간 배회)",
    "timestamp":  "2025-04-04 22:15",
    "location":   "창고 외곽",
    "detections": [
      {"class": "person", "bbox": [150, 80, 230, 360], "confidence": 0.89}
    ],
  },
  {
    "label":      "시나리오 C - 위험 (심야 침입 패턴)",
    "timestamp":  "2025-04-04 02:13",
    "location":   "창고 출입구",
    "detections": [
      {"class": "person", "bbox": [120, 80, 200, 350], "confidence": 0.91},
      {"class": "person", "bbox": [310, 95, 390, 360], "confidence": 0.87},
      {"class": "car",     "bbox": [50, 200, 280, 400], "confidence": 0.95},
    ],
  },
],

print("=" * 60)
print("  step01_llm_from_opencv.py - 최종 통합 테스트")
print("=" * 60)

for scenario in test_scenarios:
    print(f"\n📷 {scenario['label']}")
    print(f"   시각: {scenario['timestamp']} / 위치: {scenario['location']}")
    print(f"   탐지: {len(scenario['detections'])}건")
    print()

    result = analyze_frame(
        detections = scenario["detections"],
        timestamp  = scenario["timestamp"],
        location    = scenario["location"],
    )

    print_result(result)

print("\n" + "=" * 60)
print("  ✅ Part 01 실습 완료!")
print()
print("  다음 단계 (Part 02 LangChain):")
print("  → analyze_frame()을 100개 프레임에 자동 반복")
print("  → 위험 프레임만 자동 필터링")
print("  → 운영자 챗봇 (이전 분석 기억)")
print("=" * 60)

```

실행 결과 (예시):

```

=====
step01_llm_from_opencv.py - 최종 통합 테스트
=====

📷 시나리오 A - 정상 (낮 출입)
  시각: 2025-04-04 14:30 / 위치: 정문 앞
  탐지: 1건

  🟢 위험도 : 정상 (확신도: 97%)
  📄 근거   : 주간 정문 1인 출입 - 정상 패턴
  🚨 조치   : 로그 기록
  📊 토큰    : 312개

📷 시나리오 B - 주의 (야간 배회)
  시각: 2025-04-04 22:15 / 위치: 창고 외곽
  탐지: 1건

  🟡 위험도 : 주의 (확신도: 82%)
  📄 근거   : 야간 창고 외곽 1인 배회 - 이상 접근 패턴
  🚨 조치   : 경비팀 확인 요청

```

 토큰 : 334개

- 위험도 : 위험 (확신도: 94%)
- 📅 근거 : 심야 핵심 구역 2인+차량 - 침입 시도 패턴
- 🚔 조치 : 경찰 즉시 신고 + 비상 알람
- ? 확인불가: 인물 신원, 차량 내부 상황
- 📊 토큰 : 367개

✅ Part 01 실습 완료!

다음 단계 (Part 02 LangChain):

- `analyze_frame()`을 100개 프레임에 자동 반복
- 위험 프레임만 자동 필터링
- 운전자 챗봇 (이전 분석 기억)

 CH03 완료 체크리스트

- ☐ 프롬프트 4요소를 빈 종이에 설명할 수 있다
- ☐ Zero-shot / Few-shot / CoT 각각 언제 쓰는지 말할 수 있다
- ☐ `format_detections()` 함수를 직접 작성하고 실행했다

- ☐ Few-shot 예시를 정상/주의/위험 각 1개씩 직접 작성했다
 - ☐ 환각 방지 제약 조건이 왜 필요한지 설명할 수 있다
 - ☐ `step01_llm_from_opencv.py` 를 실행해서 3가지 시나리오를 분석했다
 - ☐ 결과 JSON에서 `risk_level` 값을 꺼내서 위험도별 분기 처리를 해봤다
-

다음: Part 02 · CH01 — LangChain 이해 → 반복 API 호출의 한계, Model / PromptTemplate / Chain / OutputParser 구조, LCEL 파이프라인 개념